1. Executive Summary & Design Philosophy

This architecture defines a highly optimized, enterprise-grade Retrieval-Augmented Generation (RAG) system tailored specifically for Know Your Customer (KYC) compliance.

- **Decoupled API** : Ingestion, embedding, and retrieval are exposed as an independent "Retrieval-as-a-Service" REST API. This ensures the generation layer only receives mathematically scored, factually pristine JSON payloads.
- **Algorithmic Control** : The downstream LLM is strictly treated as a reasoning engine. All domain intelligence, data filtering, and exact-matching are handled upstream by deterministic algorithms, NLP frameworks, and graph traversals.
- **Strict Auditability** : Every retrieved fact must be traceable to a specific document pixel and algorithmic decision path.

---

2. High-Level Design (HLD)

The framework operates across three distinct functional layers, storing all text, metadata, and high-dimensional vectors inside a centralized PostgreSQL database (using the `pgvector` extension).

1. **Layer 1 (Ingestion & Layout)** : Handles the physical files by using computer vision (Azure OCR) and layout-aware NLP to break documents down without destroying tabular or form-based data.
2. **Layer 2 (Semantic Chunking & Metadata)** : Prepares the data for generic embedding models and extracts strict metadata to bypass vector search entirely for exact-match queries.
3. **Layer 3 (Knowledge Graph & Audit)** : Handles multi-hop reasoning to connect disparate documents and logs the exact algorithmic decisions for compliance.

---

3. Low-Level System Design (LLD)

3.1. Layer 1: Ingestion & Layout-Aware Processing Service

Standard naive chunking destroys the structural integrity of KYC documents, separating critical context like a "Director Name" from a "Date of Birth". To resolve this, we implement an **Ingestion Router** that categorizes and parses files based on layout rather than token count.

- **Visual & Forms (IDs, PNGs)** : These files are routed to Vision-language models or specialized Document AI to extract explicit Key-Value pairs. **Integration Point:** The Azure OCR REST service will operate here, performing high-fidelity extraction on scanned KYC forms and IDs.
- **Layout-Heavy (PDF, DOCX)** : These are routed to layout-aware parsers such as unstructured.io or LayoutLM. Embedded tables are explicitly extracted and converted to Markdown or HTML to preserve row/column relationships.
- **Native Data (XLSX)** : These files are parsed via native libraries directly into structured JSON.

3.2. Layer 2: Hierarchical Chunking & NLP Service

The system requires a **Parent-Child Retrieval** pattern mapped onto a **Knowledge Graph** to satisfy the differing needs of the embedding model and the downstream LLM.

- **Parent Nodes** : These represent the broader document sections, tables, or raw image URIs.
- **Child Nodes** : These represent highly granular, descriptive text optimized for vector search.
- **Graph Edges** : These are the logical links connecting chunks, such as `CHILD_OF` or `REFERENCES_UBO`.
- **NLP Entity Extraction** : We will use specialized, open-source NLP frameworks at the ingestion layer, such as spaCy or GLiNER, for Named Entity Recognition.
- **Vector Embeddings** : We will rely on robust, generic embedding models like OpenAI's text-embedding-3-small or BAAI's bge-m3.

3.3. Layer 3: Advanced Retrieval Gateway & Intelligent Routing

Dense vector embeddings capture semantic meaning but notoriously fail at exact alphanumeric matching, which is critical for KYC data like passport numbers. When a query hits the REST API, the workflow executes a multi-layered gateway prioritizing deterministic matching over semantic guessing.

- **Query Intent Parsing** : User queries are parsed via NLP (e.g., spaCy) to identify strict entities.
- **Deterministic Bypass** : If an exact entity is detected (like `PASSPORT_NUM`), the router bypasses vector math entirely. It executes a direct SQL lookup against the metadata table to immediately pull the pristine layout segment.
- **Hybrid Search** : If no strict entity is found, the system combines Dense Vector search (HNSW index) with Sparse Lexical search (BM25 keyword weights).
- **Reciprocal Rank Fusion (RRF)** : The algorithm mathematically merges the dense and sparse scores to bubble up the best chunks. Ensure developers implement this specific merging formula:

$$RRF(d) = \sum_{r \in R} \frac{1}{k + rank_r(d)}$$

(Note: k is a smoothing constant, usually 60, and $rank_r(d)$ is the document's rank in each respective search) .

- **Cross-Encoder Reranking** : The top RRF results are scored by a Cross-Encoder model (e.g., bge-reranker-v2-m3). A dynamic confidence threshold ensures low-scoring chunks are aggressively discarded before they reach the LLM.
- **Graph Traversal** : To avoid LLM-in-the-loop latency, graph traversals are rule-based.
- *Frequency Rule:* If a predefined number of retrieved Child Nodes share the exact same Parent Node, the system drops the children and retrieves the broader Parent block.
- *Semantic Threshold Rule:* If the vector similarity score of a retrieved Child's Parent exceeds a hard threshold, the Parent is pulled into the context.

---

4. Database Schema Documentation (pgvector ERD)

To provide total transparency for compliance officers, the database schema is optimized for fast retrieval and strict logging.

- **`raw_document`** : Tracks the original file. Contains `doc_id` (UUID, PK), `customer_id` (VARCHAR, B-Tree index), `file_type` (ENUM), and the secure `storage_uri`.
- **`parsed_layout_segment`** : Stores the "Parent" structural blocks. Contains `segment_id` (UUID, PK), links to the `doc_id`, and holds the `raw_content` (JSONB) such as Markdown tables or HTML.
- **`semantic_child_chunk`** : Stores the granular "Child" nodes for dense search. Contains `chunk_id` (UUID, PK), `text_content` (TEXT, GIN Index for BM25), `dense_vector` (VECTOR with an HNSW index), and `sparse_vector` (JSONB).
- **`extracted_entity`** : Holds deterministic metadata for strict routing. Contains `entity_id` (UUID, PK), `entity_type` (ENUM like `PERSON` or `PASSPORT_NUM`), and the `entity_value` (VARCHAR, Hash index).
- **`knowledge_graph_edge`** : Defines the relationship graph. Maps `source_node` and `target_node` to a `relationship_type` (ENUM like `CHILD_OF`).
- **`retrieval_audit_log`** : Tracks the exact lifecycle of a user query. Logs the `query_id` (UUID, PK), the `router_decision` (e.g., metadata_exact_match), the final `confidence_scores`, and the `retrieved_chunks` passed to the generation phase.

---

5. Summary of the Complete System Design

If we zoom out, the complete enterprise architecture looks like this:

1. **Edge:** API Gateway handling Auth & Routing.
2. **Compute (Async):** Celery Workers interacting with Azure OCR & Embedding APIs.
3. **Compute (Sync):** High-speed Retrieval APIs executing Python/SQL logic.

4. **Data Layer:** `pgvector` (Primary + Replicas), Redis (Semantic Caching & Broker), and Object Storage (Raw Files).
5. **Security & Observability Layer:** OpenTelemetry, RLS, and WORM Audit Logs.

Focusing on the REST contracts

Exposing ingestion, embedding, and retrieval as an independent "Retrieval-as-a-Service" REST API ensures that the generation layer only receives mathematically scored, factually pristine JSON payloads, maintaining the decoupled architecture. Because we are relying on generic embeddings, the magic happens in the Query Router.

PFB the detailed REST endpoint specifications for the routing layer, mapping out the algorithmic data flow and intelligent routing.

1. Endpoint Definition: `POST /api/v1/retrieve`

This endpoint serves as a multi-layered retrieval gateway prioritizing deterministic matching over semantic guessing. This schema entirely isolates the data processing from the LLM.

1.1 Request Payload Schema

The client (or the downstream generation application) sends a standardized payload containing the user's raw query alongside identifying context to filter the search space.

JSON

```
{
  "query": "Who is the UBO for account 98765 and what is their Passport
A1234?",
  "customer_id": "cust_55938_abc",
  "top_k": 5
}
```

2. Algorithmic Data Flow (Internal API Logic)

When a query hits the REST API, the workflow should execute as follows:

- **Step 1: Query Intent & NER Parsing:** The query is parsed using the same NLP framework used during ingestion. User queries are parsed via NLP (e.g., spaCy) to identify strict entities. If the user asks for "Passport A1234," the router detects the `PASSPORT_NUM` entity.
- **Step 2: Deterministic Bypass:** If an exact entity (like `PASSPORT_NUM`) is detected, the router bypasses vector math entirely. Instead of doing a vector search, the router hits the `extracted_entity` table via standard SQL, grabs the `chunk_id`, and immediately pulls the pristine `parsed_layout_segment`.
- **Step 3: Hybrid Search:** For broader queries, the system combines Dense Vector search (HNSW index) with Sparse Lexical search (BM25 keyword weights). If no strict entity is found, it executes a dense vector search against `dense_vector` and a sparse keyword search against `text_content`.
- **Step 4: Reciprocal Rank Fusion (RRF):** The algorithm merges the dense and sparse scores to bubble up the best chunks. Ensure the developers implement this formula:

  $$RRF(d) = \sum_{r \in R} \frac{1}{k + rank_r(d)}$$

  (where k is a smoothing constant, usually 60, and $rank_r(d)$ is the document's rank in each respective search).
- **Step 5: Cross-Encoder Reranking & Graph Traversal:** The top RRF results are evaluated by a Cross-Encoder model (e.g., `bge-reranker-v2-m3`). If a child chunk scores high, the algorithm traverses the `knowledge_graph_edge` to retrieve the parent segment for full context. A dynamic confidence threshold ensures low-scoring chunks are aggressively discarded before they reach the LLM, preventing hallucinations.

3. Response Payload Schema

The final JSON payload sent to the generation layer is mathematically sound, factually pristine, and 100% auditable. It must include the exact parameters required to populate the `retrieval_audit_log`, which tracks the specific router path taken, cross-encoder confidence scores, and the final array of chunk IDs sent to the LLM.

JSON

```
{
  "query_id": "uuid-9948-abcd",
  "router_decision": "metadata_exact_match",
  "retrieved_context": [
    {
      "chunk_id": "uuid-1122-efgh",
      "segment_id": "uuid-3344-ijkl",
      "raw_content": {
        "type": "embedded_table",
        "markdown": "| Director Name | Passport |\n|---|---|\n| Jane Doe |
A1234 |"
      },
      "confidence_score": 0.98,
      "traversal_path": "direct_sql_bypass"
    }
  ],
  "audit_metadata": {
    "reranker_model": "bge-reranker-v2-m3",
    "hybrid_search_triggered": false
  }
}
```

Knowlege Graph Traversal

This is breakdown of the exact graph traversal rules designed for your KYC Retrieval-Augmented Generation system.

By pushing this intelligence into deterministic graph traversals, we ensure that multi-hop reasoning (like tracing Ultimate Beneficial Ownership across multiple documents) happens upstream. These traversal rules are specifically engineered to avoid LLM-in-the-loop latency.

Graph Traversal Rules

The system utilizes a Parent-Child Retrieval pattern mapped onto a Knowledge Graph to govern how context is gathered. The execution relies on the following strict rules:

- **Frequency Rule** : If a predefined number of retrieved Child Nodes share the exact same Parent Node, the system drops the children and retrieves the broader Parent block.

- **Semantic Threshold Rule** : The system checks the vector similarity score of a retrieved Child's Parent; if it exceeds a hard threshold, the Parent is pulled into the context.
- **Cross-Encoder Context Retrieval Rule** : Following Reciprocal Rank Fusion, the top results are scored by a Reranker. If a child chunk scores high, the algorithm traverses the knowledge graph edge to retrieve the parent segment for full context.

Knowledge Graph Edge Structure

To execute these rules efficiently via SQL or graph queries, the system maps the relationships between chunk IDs for traversal using the `knowledge_graph_edge` table.

PFB the exact schema detailing how these relationships are stored:

Column Name	Data Type & Details	NLP / Engineering Purpose
edge_id	UUID (PK)	Defines the relationship graph.
source_node	UUID (B-Tree)	Origin chunk_id or segment_id.
target_node	UUID (B-Tree)	Destination chunk_id or segment_id.
relationship_type	ENUM	E.g., CHILD_OF, SAME_ADDRESS, CONTRADICTS.

Ingestion API payload

By implementing an Ingestion Router, we categorize and parse files based on layout rather than token count. This ensures we don't destroy the structural integrity of KYC documents, avoiding issues like separating a "Director Name" from their "Date of Birth".

1. Endpoint Definition: `POST /api/v1/ingest`

This endpoint acts as the entry point for the **Retrieval-as-a-Service pipeline**. It handles physical files by routing Visual & Forms (IDs, PNGs) to the Azure OCR service to extract

explicit Key-Value pairs.

1.1 Request Payload Schema

The client sends the document URI and necessary metadata to link the file to a specific customer profile.

JSON

```
{
  "customer_id": "cust_55938_abc",
  "file_type": "png",
  "storage_uri": "s3://kyc-vault/cust_55938_abc/passport.png",
  "processing_directives": {
    "force_ocr": true,
    "ocr_provider": "azure_rest_api"
  }
}
```

1.2 Processing Logic (Internal Data Flow)

Once the payload is received, the Ingestion Router orchestrates the data through the pipeline to populate the database schema:

- **Layer 1 (Layout Analysis)** : The Azure OCR extracts the raw text and bounding boxes. The system stores this as the overarching Parent Node in the `parsed_layout_segment` table. This raw content is saved as JSONB Key-Value dicts.
- **Layer 2 (Semantic Chunking)** : The system creates highly granular, descriptive text optimized for vector search. These Child Nodes are stored in the `semantic_child_chunk` table.
- **Layer 2 (Entity Extraction)** : Deterministic metadata for strict routing (like a `PASSPORT_NUM`) is extracted and stored in the `"extracted_entity` table.

1.3 Response Payload Schema

The synchronous response confirms the successful processing of the Azure OCR data and its mapping into the hierarchical database structure.

JSON

```
{
  "doc_id": "uuid-doc-7788",
  "status": "ingested",
  "layout_segments_created": 1,
  "semantic_chunks_created": 4,
  "entities_extracted": [
    {
      "entity_type": "PASSPORT_NUM",
      "entity_value": "A1234"
    },
    {
      "entity_type": "PERSON",
      "entity_value": "John Doe"
    }
  ],
  "graph_edges_created": 4
}
```

PGVECTOR Design

Here is the comprehensive design and operational guide for the `pgvector` database and its surrounding infrastructure.

---

1. Enterprise Infrastructure & Supporting Services

To handle high-throughput KYC document ingestion while maintaining low-latency REST API retrieval, the vector database should not stand alone. It requires a robust ecosystem of supporting services.

- **Connection Pooling (PgBouncer):** Vector similarity searches keep database connections open longer than standard key-value lookups. We must place PgBouncer (or Odyssey) in front of the PostgreSQL instance to multiplex connections and prevent connection starvation during peak API traffic.
- **Background Task Queue (Celery / RabbitMQ):** Do not generate embeddings or insert vectors synchronously within the REST API endpoints. Use a worker queue to batch vector insertions. Inserting vectors in batches of 1,000 is significantly faster than inserting them one by one.

- **Semantic Cache (Redis):** Because KYC queries often repeat (e.g., standard compliance checklists), implement a caching layer. Before hitting `pgvector`, the system hashes the query intent. If a statistically identical query was recently executed, Redis serves the pre-computed RRF scores and `chunk_id` arrays directly, saving database CPU.

---

2. pgvector Indexing Strategy: HNSW Configuration

The architectural schema explicitly defines the `semantic_child_chunk` table using an HNSW (Hierarchical Navigable Small World) index for the `dense_vector` column. HNSW is the enterprise standard because it provides superior query performance and higher recall compared to IVFFlat, without requiring a training step.

To optimize the HNSW index for production, the database administrators must tune the following parameters:

- `m` **(Max Connections per Layer):** This defines how dense the graph is. The default is 16. For higher-dimensional embeddings (like 1536-dimension OpenAI vectors), increasing `m` to **24 or 32** improves accuracy at a slight cost to index build time and memory.
- `ef_construction` **(Index Build Candidate List):** This determines how many candidates are considered when adding a new vector to the graph. The default is 64. Increasing this to **100 or 200** significantly improves index quality (recall) but takes longer to build.
- `ef_search` **(Query Time Candidate List):** This is configured dynamically at the session level during a search. Higher values increase accuracy at the cost of latency.

SQL Implementation for Index Creation:

SQL

```
-- Enable the extension (idempotent)
CREATE EXTENSION IF NOT EXISTS vector;

-- Create the HNSW index on the child chunk table
CREATE INDEX idx_semantic_child_vector ON semantic_child_chunk
USING hnsw (dense_vector vector_cosine_ops)
WITH (m = 24, ef_construction = 100);
```

---

3. Query Execution & Tuning for the Router Layer

When the Advanced Retrieval Gateway executes the Hybrid Search, we must ensure the query planner uses the HNSW index properly.

- **Session-Level Tuning:** In the REST API retrieval function, right before executing the vector distance query, adjust the `ef_search` parameter for that specific transaction. **SQL**

  ```
  BEGIN;
  SET LOCAL hnsw.ef_search = 100;
  -- Execute the dense vector search here
  COMMIT;
  ```

- **Filter Before Vector Math:** If the query router identifies metadata (like a specific `customer_id`), always apply that SQL `WHERE` filter *before* executing the vector distance calculation (`<=>`). This drastically reduces the graph search space.

---

4. Memory Management & Maintenance

The single most critical factor in `pgvector` performance is memory. **the HNSW index must fit entirely into RAM.** If PostgreSQL is forced to read the graph from the disk, the API latency will spike from milliseconds to seconds.

- **Sizing the Instance:** The HNSW index is typically 2 to 3 times larger than the raw vector data. When sizing the Azure PostgreSQL instance, calculate the expected vector size and ensure the RAM is at least 3x that amount.

- **`maintenance_work_mem`:** Building an HNSW index is memory-intensive. Before running the `CREATE INDEX` command, temporarily increase the maintenance memory to speed up the build process: **SQL**

  ```
  SET maintenance_work_mem = '4GB'; -- Adjust based on the instance size
  ```

By wrapping `pgvector` in a robust task queue, aggressively caching with Redis, and precisely tuning the HNSW graph parameters, the retrieval engine will easily meet the demands of enterprise-grade KYC compliance.

PFB precise database queries for the **Deterministic Bypass** and **Hybrid Search** phases, utilizing PostgreSQL and the `pgvector` extension.

---

1. The Deterministic Bypass (Exact Match SQL)

The goal of this phase is to prioritize absolute accuracy. If a strict entity (like a passport number) is detected by your NLP parser, the router bypasses vector math entirely. Instead of guessing via semantic similarity, it executes a direct SQL lookup to immediately pull the pristine layout segment.

This query joins the `extracted_entity` table with the `semantic_child_chunk` to find the routing link, and finally pulls the raw JSONB content from the `parsed_layout_segment`.

SQL

```
-- Query 1: Deterministic Bypass for 'PASSPORT_NUM'
SELECT
    pls.segment_id,
    pls.doc_id,
    pls.raw_content, -- Retrieves the pristine Markdown/HTML table
    ee.entity_type,
    ee.entity_value
FROM
    extracted_entity ee
JOIN
    semantic_child_chunk scc ON ee.chunk_id = scc.chunk_id
JOIN
    parsed_layout_segment pls ON scc.segment_id = pls.segment_id
WHERE
    ee.entity_type = 'PASSPORT_NUM'
    AND ee.entity_value = 'A1234'  -- Injected by your Query Intent Parser
LIMIT 1;
```

---

2. Hybrid Search with Reciprocal Rank Fusion (pgvector)

When a query is broader and no strict entity is found, the system falls back to a **Hybrid Search** . This combines a Dense Vector search (using `pgvector`'s HNSW index) with a Sparse Lexical search (using PostgreSQL's native full-text search as our BM25 equivalent).

To bubble up the best chunks, the algorithm merges the dense and sparse scores using the Reciprocal Rank Fusion (RRF) formula:

$$RRF(d) = \sum_{r \in R} \frac{1}{k + rank_r(d)}$$

(Where k is a smoothing constant, usually set to 60) .

This query uses Common Table Expressions (CTEs) to perform both searches independently, rank them, and mathematically fuse the results.

SQL

```
-- Query 2: Hybrid Search (Dense pgvector + Sparse Lexical) with RRF
WITH dense_search AS (
    -- Step A: Dense Vector Search using pgvector cosine distance (<=>)
    SELECT
        chunk_id,
        segment_id,
        text_content,
        -- The array below is the embedding generated from the user's query
        dense_vector <=> '[0.015, -0.022, 0.081, ...]'::vector AS
vector_distance,
        ROW_NUMBER() OVER (ORDER BY dense_vector <=> '[0.015, -0.022, 0.081,
...]'::vector ASC) AS dense_rank
    FROM
        semantic_child_chunk
    ORDER BY
        vector_distance ASC
    LIMIT 60
),
sparse_search AS (
    -- Step B: Sparse Lexical Search (PostgreSQL Full-Text Search)
    SELECT
        chunk_id,
        -- Calculates term frequency/relevance score
        ts_rank_cd(to_tsvector('english', text_content),
plainto_tsquery('english', 'UBO shareholder stake')) AS sparse_score,
        ROW_NUMBER() OVER (ORDER BY ts_rank_cd(to_tsvector('english',
```

```
text_content), plainto_tsquery('english', 'UBO shareholder stake')) DESC) AS
sparse_rank
    FROM
        semantic_child_chunk
    WHERE
        to_tsvector('english', text_content) @@ plainto_tsquery('english',
'UBO shareholder stake')
    ORDER BY
        sparse_score DESC
    LIMIT 60
)
-- Step C: Reciprocal Rank Fusion (RRF)
SELECT
    COALESCE(d.chunk_id, s.chunk_id) AS chunk_id,
    COALESCE(d.segment_id, s.segment_id) AS segment_id,
    COALESCE(d.text_content, s.text_content) AS text_content,
    -- Calculate RRF Score with k=60
    COALESCE(1.0 / (60 + d.dense_rank), 0.0) +
    COALESCE(1.0 / (60 + s.sparse_rank), 0.0) AS rrf_score
FROM
    dense_search d
FULL OUTER JOIN
    sparse_search s ON d.chunk_id = s.chunk_id
ORDER BY
    rrf_score DESC
LIMIT 5; -- Top K results passed to the Cross-Encoder Reranker
```

Python workflows and Batch Processes

PFB detailed Python and Celery worker architecture designed to handle the batch embedding pipeline for KYC RAG framework.

---

1. The Asynchronous Embedding Workflow

The batch embedding pipeline decouples the slow process of vector math from the fast process of document ingestion.

- **Ingestion API (FastAPI/Flask):** Receives the document, creates a record in the `raw_document` table, and pushes a task message to the broker.
- **Message Broker (Redis/RabbitMQ):** Holds the queue of chunking and embedding tasks.
- **Celery Workers (Python):** Independent background processes that consume tasks from the queue.

- **Processing:** The workers use specialized NLP frameworks like unstructured.io or LayoutLM for layout parsing, and spaCy or GLiNER for Named Entity Recognition.
- **Batch Embedding:** The workers group the granular child nodes and send them in batches to generic models like OpenAI's text-embedding-3-small or BAAI's bge-m3.
- **Database Insertion:** The generated vectors are batch-inserted into the `semantic_child_chunk` table alongside the clean text indexed for BM25/keyword search.

---

2. Python & Celery Implementation Strategy

Here is a blueprint of how the engineering team should structure the Celery tasks to ensure high throughput and prevent database locking.

A. Task Definition & NLP Parsing

First, the worker pulls the document and performs the heavy layout parsing and chunking.

Python

```
from celery import shared_task
from typing import List
import db_client # the internal pgvector DB client
import nlp_parser # the internal wrapper for unstructured.io / spaCy

@shared_task(bind=True, max_retries=3)
def process_kyc_document_task(self, doc_id: str, storage_uri: str):
    try:
        # 1. Layout-Aware Parsing (e.g., using unstructured.io)
        # Extracts overarching semantic summaries as Parent nodes
        layout_segments = nlp_parser.extract_layout(storage_uri)
        db_client.insert_parsed_layout_segments(doc_id, layout_segments)

        # 2. Semantic Chunking & Entity Extraction (e.g., using
spaCy/GLiNER)
        # Creates granular Child nodes
        child_chunks =
nlp_parser.chunk_and_extract_entities(layout_segments)

        # 3. Route to Batch Embedding
        # We don't embed one by one; we pass the list of chunks to the next
phase
        batch_embed_and_store.delay(child_chunks)

    except Exception as exc:
        self.retry(exc=exc, countdown=60) # Exponential backoff recommended
```

B. Batch Embedding & Database Insertion

To avoid rate-limiting the embedding provider and to optimize PostgreSQL write speeds, vectors must be generated and inserted in batches.

Python

```
from embedding_provider import get_embeddings # Wrapper for OpenAI or bge-m3

@shared_task
def batch_embed_and_store(child_chunks: List[dict]):
    # 1. Isolate text for batch embedding
    texts_to_embed = [chunk['text_content'] for chunk in child_chunks]

    # 2. Call the embedding model (e.g., text-embedding-3-small) in one
batch
    # This is significantly faster than looping HTTP requests
    embeddings = get_embeddings(texts_to_embed)

    # 3. Zip the embeddings back into the chunk dictionaries
    for chunk, vector in zip(child_chunks, embeddings):
        chunk['dense_vector'] = vector

    # 4. Batch Insert into pgvector
    # Use psycopg3's execute_values or SQLAlchemy's bulk_insert_mappings
    db_client.bulk_insert_semantic_child_chunks(child_chunks)
```

---

3. Operational Guardrails for Production

To ensure this pipeline remains stable under the load of thousands of KYC documents:

- **Batch Sizes:** When calling the embedding model, keep batch sizes between 500 and 1,000 texts. Going too large may result in API timeouts from the provider; going too small wastes network overhead.
- **Database Write Locks:** When writing to the `semantic_child_chunk` table where the pgvector HNSW index lives, bulk inserts are mandatory. Inserting vectors row-by-row will heavily fragment the index and slow down ingestion.
- **Idempotency:** Ensure the Celery tasks are idempotent. If a worker crashes mid-embedding, rerunning the task should not result in duplicate child chunks or corrupted knowledge graph edges. Use the `chunk_id` (UUID) to enforce `ON CONFLICT DO UPDATE` logic in the SQL inserts.

Here are the remaining architectural components and designs that need to be implemented in the Future. [Not focussing on these for Now]

Pillar 1: Security, Compliance & Data Isolation (KYC Critical)

Because this system processes passports, financial histories, and UBO (Ultimate Beneficial Owner) declarations, standard database security is insufficient.

- **RAG Access Control (Chunk-Level Security):** Security cannot rely solely on the UI hiding data. The vector database must enforce entitlements. We must implement **Row-Level Security (RLS)** in PostgreSQL. When a query hits the `/api/v1/retrieve` endpoint, it must pass the user's IAM role or tenant ID, ensuring the `semantic_child_chunk` table only returns vectors the user is legally authorized to see.
- **PII Redaction at Ingestion:** Before sending visual data to the Azure OCR service or text to the embedding model (like OpenAI), sensitive entities (like exact SSNs or raw account numbers) should be dynamically masked.
- **Data at Rest & Transit:** The `raw_document` storage (e.g., Azure Blob Storage or AWS S3) must use AES-256 encryption. The connection between the Application layer, Azure OCR, and `pgvector` must be strictly enforced via TLS 1.3 within a private VPC/VNet.

Pillar 2: Scalability & High Availability (HA)

Vector math is extremely CPU-bound. If a sudden influx of KYC documents arrives during a compliance audit, the system must not buckle.

- **pgvector Read-Replica Strategy:** Vector searches (the `SELECT` queries doing cosine distance math) will consume the primary database's CPU. We must implement a Primary-Replica architecture. The Celery ingestion workers write embeddings to the Primary DB, while the REST API routes all `<=>` vector similarity searches to a pool of Read Replicas.
- **API Gateway & Rate Limiting:** The Retrieval-as-a-Service REST APIs must sit behind an API Gateway (like Kong, Apigee, or Azure API Management). This

enforces JWT token validation, handles SSL termination, and throttles requests to prevent noisy-neighbor issues if one client spams the retrieval service.

- **Asynchronous OCR Fallbacks:** Azure OCR has rate limits. the Celery worker queue must implement exponential backoff and a Dead Letter Queue (DLQ). If Azure OCR goes down or throttles the requests, the ingestion tasks are safely parked in the DLQ to be retried later without losing customer data.

Pillar 3: Observability, Evaluation & Auditing

In production, We cannot manually check if the retrieval system is fetching the right documents. We need automated metrics.

- **Distributed Tracing (OpenTelemetry):** Every request entering the API gets a unique `trace_id`. This ID tracks the latency of the NLP parser, the Azure OCR response time, the pgvector query time, and the Cross-Encoder reranking time. If an API call takes 4 seconds, the engineers will know exactly which microservice caused the bottleneck.
- **RAG-Specific KPIs:** Beyond basic CPU/RAM monitoring, the system must track AI-specific metrics:
  - *Retrieval Coverage:* How often do queries return zero confident chunks?
  - *Reranker Drop Rate:* How many chunks are surviving the Cross-Encoder threshold?
- **Immutable Audit Trails:** We designed the `retrieval_audit_log` table, but for compliance, this data should periodically be flushed to a WORM (Write Once, Read Many) storage system like AWS S3 Glacier or Azure Blob Immutable Storage. This proves to auditors exactly what data was retrieved for a specific KYC decision, immune to tampering.
